Explanation of the config_random.yaml file				
key_or_index	value	type	Explanation	Notes
Setup for the graph				
mode	random or manual	str	This indicates if you want to manually add all the initializtion parameters. Leave it as random.	
num_nodes	10	int	Number of nodes that we want in the graph	
graph_model	barabasi_albert	str	The way we want to generate the graph. The values can be erdos_renyi (p), barabasi_albert (m), watts_strogatz (k, p), balance_tree (r, h)	The Barabasi-Albert (BA) model generates "scale-free" networks, which are the most realistic representation of the broad Computing Continuum.
m	4	int	Parameter of the Barabasi Albert model	
disable_random_node	Distribution = random.uniform(0, 300)	str	Defines how often you want to disable a random node	
	distribution_to_enable_node: 'random.uniform(0, 100)'	str	When we call disable_random_node it automatically adds an event to revive the node and in here we defines how often we want to revive that node that we have disabled	
	sim_set	str	Value needed for the random values that need a random seed	
disable_random_edge	Distribution = random.uniform(0, 300)	str	Defines how often you want to disable a random edge	
	distribution_to_enable_edge: 'random.uniform(0, 100)'	str	When we call disable_random_edge it automatically adds an event to revive the edge and in here we defines how often we want to revive that edge that we have disabled	
	sim_set	str	Value needed for the random values that need a random seed	
node	ram: 'rng.pareto(1.16, {num_nodes})'	str	RAM of the node	We make sure that the nodes that have the biggest RAM act like "hubs". We keep {num_nodes} so the parser can format it via kwargs.
	min_ram: 8.0	fl	Smallest amount of RAM possible for a node	
	max_ram: 16.0	fl	Biggest amount of RAM possible for a node	
edge	delay: 'random.uniform(0.1, 5.0)'	str	Delay on one edge	
Apps				
num_apps	20	int	How many apps we start with	These values are used for initalizing the simulation. We can either choose the number of apps or the saturation.
saturation_percentage	80	fl	The percentage of saturation/occupation desired in the total graph	These values are used for initalizing the simulation. We can either choose the number of apps or the saturation.
popularity	random.uniform(0.1, 1.0)	str		
cpu	random.uniform(0.1, 4.0)	str		
ram	random.uniform(0.5, 8.0)	str		

disk	random.uniform(10, 100)	str		
time	random.uniform(0, 300)	str		
num_new_users	rng.integers(1, 6)'	str	Every time an app gets created at the beginning of the simualtion a random amount of users (between one and six) get associated to this app	
remove_app	Distribution: random.uniform(0, 300)	str		
increase_popularity	distribution: 'random.uniform(0, 300)'	str		
	multiplier: 'random.uniform(1, 3)'		The multiplier that the current request_ratio of one specific user is multiplied for	
	sim_set	str	Value needed for the random values that need a random seed	
decrease_popularity	distribution: 'random.uniform(0, 300)'	str		
	multiplier: 'random.uniform(0, 1)'		The multiplier that the current request_ratio of one specific user is multiplied for	
	sim_set	str	Value needed for the random values that need a random seed	
Users				
num_users	2	int	How many users we start with	This value is not used anymore
request_ratio	rng.exponential(5)'	str		The inter-arrival time between consecutive app requests from a single user follows an **exponential distribution**. You would assign a rate parameter lambda (e.g., 5 requests per minute) to generate these temporal timestamps in your simulation.
request_popularity	random.uniform(0.1, 1.0)	str		
centrality	random.uniform(0.1, 1.0)	str		
remove_user	Distribution: 'rng.exponential(300)'	str	Users stay for 300 time units on average	The time a user spends in either state (new or remove) before transitioning to the other is modeled using an exponential distribution. On p. 7 of the paper: “we considered applications with a mean active/sleeping time (1/λ) of 300 seconds”
move_user	Distribution: 'rng.exponential(120)'	str	Users move approximately every 120 units of time	In Random Waypoint (RWP) mobility model, a user randomly selects a destination node in the network and moves toward it at a speed chosen uniformly from a predefined range (e.g., a uniform distribution between 0 and 50 m/s).
	sim_set	str	Value needed for the random values that need a random seed	
increase_request_ratio	distribution: 'rng.exponential(90)'	str	Each user will increase its ratio approximately every 90 units	
	multiplier: 'random.uniform(1, 3)'	str	The multiplier that the current request_ratio of one specific user is multiplied for	Multiplies by a random value between 1 and 3
	sim_set	str	Value needed for the random values that need a random seed	
decrease_request_ratio	distribution: 'rng.exponential(90)'	str	Each user will decrease its ratio approximately every 90 units	

	multiplier: 'random.uniform(0.2, 1)'	str	The multiplier that the current request_ratio of one specific user is multiplied for	Multiplies by a random value between 0.2 and 1
	sim_set	str	Value needed for the random values that need a random seed	
New objects				
new_user	distribution: 'random.uniform(0, 300)'	str		The time a user spends in either state (new or remove) before transitioning to the other is modeled using an exponential distribution. On p. 7 of the paper: “we considered applications with a mean active/sleeping time (1/λ) of 300 seconds”
	config	dict	Whole initial configuration from the YAML	
	sim_set	str	Value needed for the random values that need a random seed	
new_app	distribution: 'rng.exponential(50)'	str		**For now this value in commented on the YAML since we don't want to modify the new apps that appear in the simulation**
	config	dict	Whole initial configuration from the YAML	
	sim_set	str	Value needed for the random values that need a random seed	
	num_new_users: 'random.randint(1, 5)'	str	When we create a new app we also create some random users associated to that new app	
Additional information				
We don’t want to modify the apps/nodes/edges for now				

Bibliographic references

Naryshkin, A. A., et al. (2021). "Monte Carlo Approach to the Computational Capacities Analysis of the Computing Continuum." *International Conference on Computational Science* — monte_carlo_approach.pdf (page 5)

Massa, J., et al. (2024). "Evaluation and Efficiency Comparison of Evolutionary Algorithms for Service Placement Optimization in Fog Architectures." arXiv preprint. —

Huang, S., Lv, B., Wang, R., & Huang, K. (2020). Scheduling for mobile edge computing with random user arrivals: An approximate MDP and reinforcement learning approach. *arXiv* . https://doi.org/10.48550/arxiv.2004.07553 - huang-pdf

Donassolo, B., Legrand, A., Mertikopoulos, P., & Fajjari, I. (2022). Online reconfiguration of IoT applications in the fog: The information-coordination trade-off. *IEEE Transactions on Parallel and Distributed Systems* , *33* , 1156–1172. https://doi.org/10.1109/tpds.2021.3097281 - donassolo.pdf page 7

Zhang, Y., Liang, W., Xu, Z., & Jia, X. (2024). Mobility-aware service provisioning in edge computing via digital twin replica placements. *IEEE Transactions on Mobile Computing* , *23* , 11295–11311. https://doi.org/10.1109/tmc.2024.3394839

Donassolo, B., Legrand, A., Mertikopoulos, P., & Fajjari, I. (2022). Online reconfiguration of IoT applications in the fog: The information-coordination trade-off. *IEEE Transactions on Parallel and Distributed Systems* , *33* , 1156–1172. https://doi.org/10.1109/tpds.2021.3097281 - donassolo.pdf page 7